

← Go Back

Hardware

Portenta Breakout

Tutorials

Using the Segger J-Link
Debugger with the Portenta
Breakout

Getting Started With the
Arduino Portenta
Breakout

Home / Hardware / Portenta Breakout / **Getting Started With the Arduino Portenta Breakout**

ON THIS PAGE

Overview

Goals

Instructions

Conclusion

Getting Started With the Arduino Portenta Breakout

This tutorial will give you an overview of the core features of the Portenta Breakout, setup the development environment and introduce the APIs required to program the board.

Author · Manuel Zomer, Pablo Marquínez, Sebastian Romero Last revision · 09/25/2024

Overview

The Arduino Portenta Breakout is a versatile tool designed for developing, testing and debugging on Portenta family boards. In this tutorial you will set up the development environment for the Portenta Breakout and learn some of its simple functionalities using the **Breakout Carrier Library**.

Goals

- About the Portenta Breakout's layout and functions
- How to setup the development environment
- How to use the `Arduino_PortentaBreakout` library
- Control an external LED using one of the carriers PWM pins
- Read an external potentiometer using an analog pin on the carrier

Required Hardware and Software

[Arduino Portenta H7](#)

[Arduino Portenta Breakout](#)

[USB-C® cable \(USB-C® to USB-A cable\)](#)

Breadboard & Cables

Pin headers for the Portenta Breakout

LED & Resistor

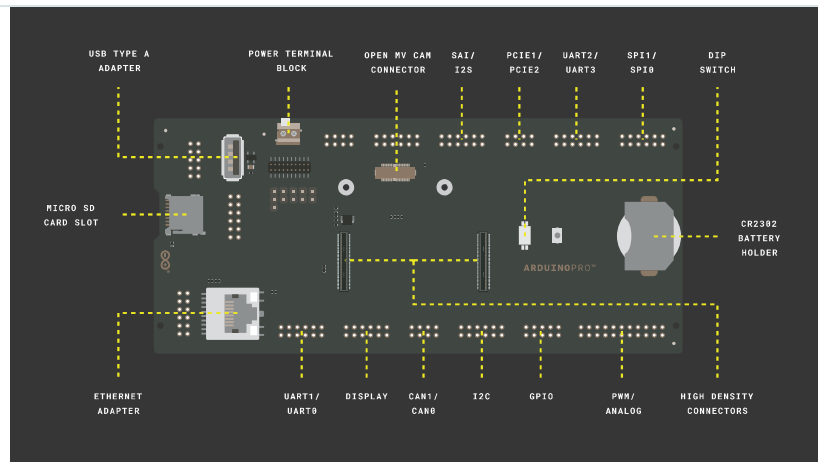
Potentiometer

[Arduino IDE 1.8.10+](#) or [Arduino IDE 2.0+](#)

Instructions

1. Get to Know the Carrier

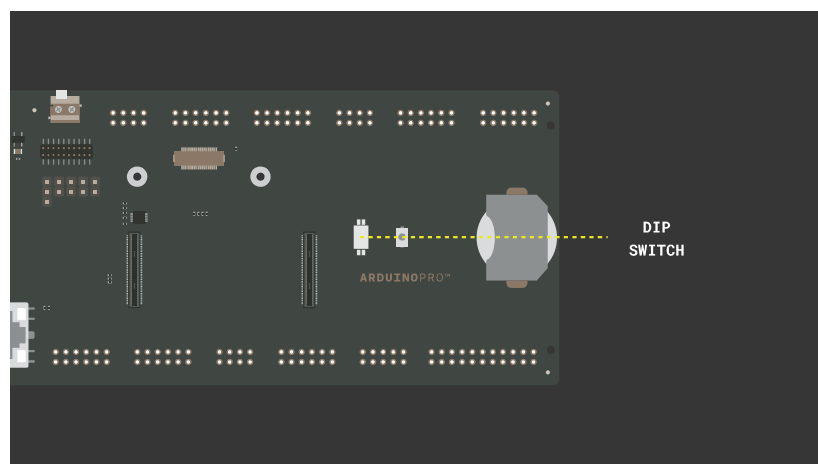
The Portenta Breakout is designed to reduce development time. It gives access to the Portenta's High Density connectors as well as features Ethernet, USB-A, a JTAG connector and more. However, for this tutorial, we will focus on using the **Arduino_PortentaBreakout** library to get access to the High Density connectors and create some simple example use cases.



2. The Basic Setup

As part of this tutorial, we will create two sample use cases for the **Arduino_PortentaBreakout** library: starting by using the Portenta Breakout's PWM pins to connect an LED and fade its brightness high and low. Secondly, we will connect an external potentiometer to showcase how to use the Portenta Breakout to read an analog input. In order to use the pins on the Portenta Breakout to connect external components, pin headers have to be soldered on to the used pins. We recommend using double row pin headers.

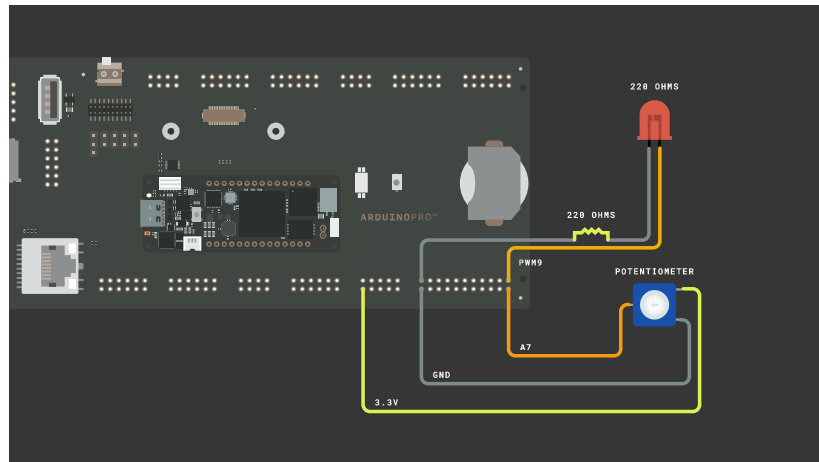
After having prepared the Portenta Breakout by soldering on pin headers, we can start using it. The board comes with two DIP switches called **BOOT** and **BT_SEL**. These switches are used to keep the Portenta in either boot-mode (BT_SEL switch) or to enable the embedded bootloader (BOOT switch) to upload firmware via the USB port on the Portenta Breakout. In this case, we have to make sure that both DIP switches are set in the **OFF** position. Having done that, we can connect the Portenta H7 to the Portenta Breakout using the two high density connectors in the orientation showcased by the dashed line on the Portenta Breakout.



3. The Circuit

In order to build this example circuit, we need our Portenta Breakout with the Portenta H7 on top and headers soldered on (at least within the ANALOG/PWM

potentiometer. To connect all these components we use jumper wires and a breadboard by following this schematics:



For the LED we can use any of the Portenta Breakout's 10 PWM Pins, in this case **PWM 9**. For the potentiometer, on the other hand, we can use one of the analog pins (A0 to A7) in order to read the potentiometer current value, in this example we use **A7**. The potentiometer also needs a 3.3V power source, which we take from the GPIO section on the Portenta Breakout, considering it being located most conveniently and close by. Eventually, potentiometer and LED have to be connected to GND to finalize the circuit.

After having connected everything, the Portenta H7 can be plugged into the computer using a USB-C® cable and we can start with the code.

4. The Arduino_PortentaBreakout Library

In the Arduino IDE we create a new Sketch and make sure we have selected the Arduino Portenta H7 on the M7 core. If you haven't used the Portenta H7 before, [here](#) you can find a detailed tutorial on how to get started with it.

In order to use the pins on the Portenta Breakout we need to install the **Arduino_PortentaBreakout** Library, which allows us to address all the pins located on the Carrier. Therefore, we need to download the library using the library manager, by going to **Sketch > Include Libraries > Manage Libraries** and search for **Arduino_PortentaBreakout**.

Once we have installed the Portenta Breakout library, we can import it to our Sketch.

```
1 #include <Arduino_PortentaBreakout.h>
```

The library defines a number of constants that can be used to call desired pins, so are PWM pins for example called `PWM0`, `PWM1`, etc and analog pins are named `ANALOG_A0`, `ANALOG_A1`, etc. A complete list with all keywords can be found below in the "Next Steps" section.

We can now create a variable of the type `breakoutPin` and thereby specify the

use an array to save a selection of pins that we can then use throughout our sketch.

```
1 breakoutPin pwmPins[] = {PWM0, PWM1, PWM2, PWM3, PWM4 , PWM5};
2 breakoutPin analogPins[] = {ANALOG_A0, ANALOG_A1, ANALOG_A2,
```

5. PWM & LED

Once we have the pins in place, we can start controlling the LED we connected to `PWM9` by creating a byte variable containing our pin number and setting the `pinMode` within the `setup()` function.

```
1 byte pwmPinNumber = 9;
2
3 void setup() {
4   pinMode(pwmPins[pwmPinNumber], OUTPUT);
5 }
```

The setup for using the LED is thereby complete, next is creating the `loop()` function, which as a first test contains two for loops to fade the LED's brightness up and down.

```
1 void loop() {
2   for (int i = 0 ; i < 255; i++) {
3     Breakout.analogWrite(pwmPins[pwmPinNumber], i);
4     delay(5);
5   }
6   for (int i = 255; i > 0; i--) {
7     Breakout.analogWrite(pwmPins[pwmPinNumber], i);
8     delay(5);
9   }
10 }
```

While uploading the sketch to the board, we should see the LED fading up and down in brightness. If that works as expected, we can proceed connecting the potentiometer to our code.

Analog Inputs Through the Portenta Breakout

Similar to the PWM code we need to define our pinNumber first, we will also create a variable to save the last value potentiometer value that has been read. Before the `setup()` function we therefore specify the following:

Within the `setup()` function we can then initialize the serial connection so we can write the incoming value to the serial monitor.

```
1 Serial.begin(9600);
2 while (!Serial);
3 Serial.println("Initialized Breakout Carrier example sketch")
```

Once we completed the setup, we can then update the loop `loop()` function to read the incoming potentiometer value and write it to the Serial Monitor. We can also update the PWM setting for the LED instead of fading up and down on repeat, to follow the potentiometers value.

```
1 void loop() {
2   uint32_t reading = Breakout.analogRead(analogPins[analogPin]);
3
4   if (reading != lastPotentiometerValue) {
5     Serial.print("Potentiometer Value: ");
6     Serial.println(reading);
7     lastPotentiometerValue = reading;
8   }
9
10  uint32_t ledValue = map(reading, 0, 1023, 0, 255);
11  Breakout.analogWrite(pwmPins[pwmPinNumber], ledValue);
12 }
```

Once the sketch is re-uploaded and we start the Serial Monitor, the Portenta starts reading the current potentiometer value and translates it into different brightness levels of our LED.

Conclusion

This sketch shows a simple usage of the Portenta Breakout in combination with the **Arduino_PortentaBreakout** Library to access and control some of the Portenta's High Density connectors. The library can thereby also be used for other protocols and pins such as I2C, UART and more. Within the "Next Steps" section below, there is a table of reference regarding how to address specific pins or what API to use.

Complete Sketch

```
1 #include <Arduino_PortentaBreakout.h>
2
3 breakoutPin pwmPins[] = {PWM0, PWM1, PWM2, PWM3, PWM4 , PI
4 breakoutPin analogPins[] = {ANALOG_A0, ANALOG_A1, ANALOG_
5
6 byte pwmPinNumber = 9;
7 byte analogPinNumber = 7;
8 uint32_t lastPotentiometerValue = -1;
9
10 void setup() {
11     pinMode(pwmPins[pwmPinNumber], OUTPUT);
12     Serial.begin(9600);
13     while (!Serial);
14     Serial.println("Initialized Breakout Carrier example sk
15 }
16
17 void loop() {
18     uint32_t reading = Breakout.analogRead(analogPins[analogPinNumber]);
19
20     if (reading != lastPotentiometerValue) {
21         Serial.print("Potentiometer Value: ");
22         Serial.println(reading);
23         lastPotentiometerValue = reading;
24     }
25
26     uint32_t ledValue = map(reading, 0, 1023, 0, 255);
27     Breakout.analogWrite(pwmPins[pwmPinNumber], ledValue);
28 }
```

Peripherals Table

This section includes a table of reference regarding the **Arduino_PortentaBreakout** Library's current peripheral names and APIs.

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
ETHERNET					
	D+	ETHERNET_DP			Not available as GPIO
	D-	ETHERNET_DN			Not available as GPIO
	GND				
	GND				
	L2	ETHERNET_L2			Not available as GPIO
	L1	ETHERNET_L1			Not available as GPIO
	C-	ETHERNET_CN			Not available

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	C+	ETHERNET_CP			Not available as GPIO
	B-	ETHERNET_BN			Not available as GPIO
	B+	ETHERNET_BP			Not available as GPIO
	A-	ETHERNET_AN			Not available as GPIO
	A+	ETHERNET_AP			Not available as GPIO
UART0				Breakout.UART0	HW flow control not supported
	3V3				
	TX	UART0_TX	PA_0		
	RX	UART0_RX	PI_9		
	RTS	UART0_RTS	PI_10		
	CTS	UART0_CTS	PI_13		
	GND				
UART1				Breakout.UART1	HW flow control not supported
	3V3				
	TX	UART1_TX	PA_9		
	RX	UART1_RX	PA_10		
	RTS	UART1_RTS	PI_14		
	CTS	UART1_CTS	PI_15		
	GND				
DISPLAY					
	D3P	DISPLAY_D3P			Not available as GPIO
	D3N	DISPLAY_D3N			Not available as GPIO
	D2P	DISPLAY_D2P			Not

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
					as GPIO
	D2N	DISPLAY_D2N			Not available as GPIO
	D1P	DISPLAY_D1P			Not available as GPIO
	D1N	DISPLAY_D1N			Not available as GPIO
	D0P	DISPLAY_D0P			Not available as GPIO
	D0N	DISPLAY_D0N			Not available as GPIO
	CLKP	DISPLAY_CLK_P			Not available as GPIO
	CLKN	DISPLAY_CLK_N			Not available as GPIO
	GND				
	GND				
CAN0					Not connected
	5V				
	TX	CAN0_TX			
	RX	CAN0_RX			
	GND				
CAN1					
	5V				
	TX	CAN1_TX	PH_13		
	RX	CAN1_RX	PB_8		
	GND				
I2C					
	3V3			Breakout.I2C_1	
	GND				
	I2C1SDA	I2C_SDA_1	PB_7		
	I2C1SCL	I2C_SCL_1	PB_6		
	3V3			Breakout.I2C_0	
	GND				

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	I2C0SCL	I2C_SCL_0	PH_7		
	3V3			Breakout.I2C_2	
	GND				
	I2C2SDA	I2C_SDA_2	PH_12		
	I2C2SCL	I2C_SCL_2	PH_11		
GPIO				Breakout.digitalWrite	
	3V3				
	0	GPIO_0	PC_13		
	1	GPIO_1	PC_15		
	2	GPIO_2	PD_4		
	3	GPIO_3	PD_5		
	4	GPIO_4	PE_3		
	5	GPIO_5	PG_3		
	6	GPIO_6	PG_10		
	GND				
	GND				
ANALOG				Breakout.analogRead	
	GND				
	REFN	ANALOG_REFN			
	REFP	ANALOG_REFP			
	A0	ANALOG_A0	PA_0_C		
	A1	ANALOG_A1	PA_1_C		
	A2	ANALOG_A2	PC_2_C		
	A3	ANALOG_A3	PC_3_C		
	A4	ANALOG_A4	PC_2		As GPIO internally connected to A2
	A5	ANALOG_A5	PC_3		As GPIO internally connected to A3
	A6	ANALOG_A6	PA_4		
	A7	ANALOG_A7	PA_6		
PWM				Breakout.analogWrite	Minimum frequency 770Hz
	GND				
	PWM0	PWM0	PA_8		
	PWM1	PWM1	PC_6		

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	PWM3	PWM3	PG_7		
	PWM4	PWM4	PJ_11		Shares timer with PWM8
	PWM5	PWM5	PK_1		
	PWM6	PWM6	PH_15		
	PWM7	PWM7	PJ_7		
	PWM8	PWM8	PJ_10		Shares timer with PWM4
	PWM9	PWM9	PH_6		
SPI0					Not connected
	3V3				
	CS	SPI0_CS			
	CK	SPI0_CK			
	CIPO	SPI0_CIPO			
	COPI	SPI0_COPI			
	GND				
SPI1				Breakout.SPI_1	
	3V3				
	CS	SPI1_CS	PI_0		
	CK	SPI1_CK	PI_1		
	CIPO	SPI1_CIPO	PC_2		
	COPI	SPI1_COPI	PC_3		
	GND				
UART2				Breakout.UART2	
	3V3				
	TX	UART2_TX	PG_14		
	RX	UART2_RX	PG_9		
	RTS	UART2_RTS			Not connected
	CTS	UART2_CTS			Not connected
	GND				
UART3				Breakout.UART3	
	3V3				
	TX	UART3_TX	PJ_8		
	RX	UART3_RX	PJ_9		

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	CTS	UART3_CTS			Not connected
	GND				
PCIE					Not connected
	TXN	PCIE_TXN			
	TXP	PCIE_TXP			
	RXN	PCIE_RXN			
	RXP	PCIE_RXP			
	CKN	PCIE_CKN			
	CKP	PCIE_CKP			
	GND				
	RST				
SAI					
	GND				
	D0	SAI_D0	PI_6		
	D1	SAI_D1			Not connected
	FS	SAI_FS	PI_7		
	SCK	SAI_SCK	PI_5		
	3V3				
I2S					
	GND				
	SDO	I2S_SDO	PI_3		
	SDI	I2S_SDI	PI_2		
	WS	I2S_WS	PB_9		
	CK	I2S_CK	PD_3		
	3V3				
CAMERA					
	D0P	CAMERA_D0P	PH_10		
	D0N	CAMERA_D0N	PH_9		
	D1P	CAMERA_D1P	PH_12		
	D1N	CAMERA_D1N	PH_11		
	D2P	CAMERA_D2P	PI_4		
	D2N	CAMERA_D2N	PH_14		
	D3P	CAMERA_D3P	PI_7		
	D3N	CAMERA_D3N	PI_6		
	CKP	CAMERA_CKP	PI_5		

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	GND				
PDM				Breakout.PDM	
	GND				
	D0	PDM_D0			
	D1	PDM_D1			Not connected
	CK	PDM_CK			
SPDIF					Not connected
	GND				
	GND				
	RX	SPDIF_RX			
	TX	SPDIF_TX			
USB0					
	GND				
	ID	USB0_ID			Not connected
	D-	USB0_DN	PA_12		
	D+	USB0_DP	PA_11		
	VBUS				
USB1					
	GND				
	ID	USB1_ID	PJ_6		
	D-	USB1_DN			
	D+	USB1_DP			
	VBUS				
SD CARD					Available as CORE peripheral
	RST				Not connected
	GND				
	WP	SD_WP			Not connected
	NC				
	CD	SD_CD			Not connected
	D0	SD_D0	PB_14		
	D3	SD_D3	PB_4		
	CMD	SD_CMD	PD_7		

PERIPHERAL	SILK	BREAKOUT	uC PIN	API	NOTES
	CLK	SD_CLK	PD_6		
	D1	SD_D1	PB_15		
	VSD	SD_VSD			
RTC				Breakout.RTCLock	

Next Steps

More examples on how to use the Portenta Breakout can be found in the examples menu in the IDE after installing the library. They can be found under **File > Examples > Arduino_PortentaBreakout**.

Suggest changes

The content on docs.arduino.cc is facilitated through a public [GitHub repository](#). If you see anything wrong, you can edit this page [here](#).

Need support?

[Help Center](#)
[Ask the Arduino Forum](#)
[Discover Arduino Discord](#)

License

The Arduino documentation is licensed under the [Creative Commons Attribution-Share Alike 4.0](#) license.

Was this article helpful?

